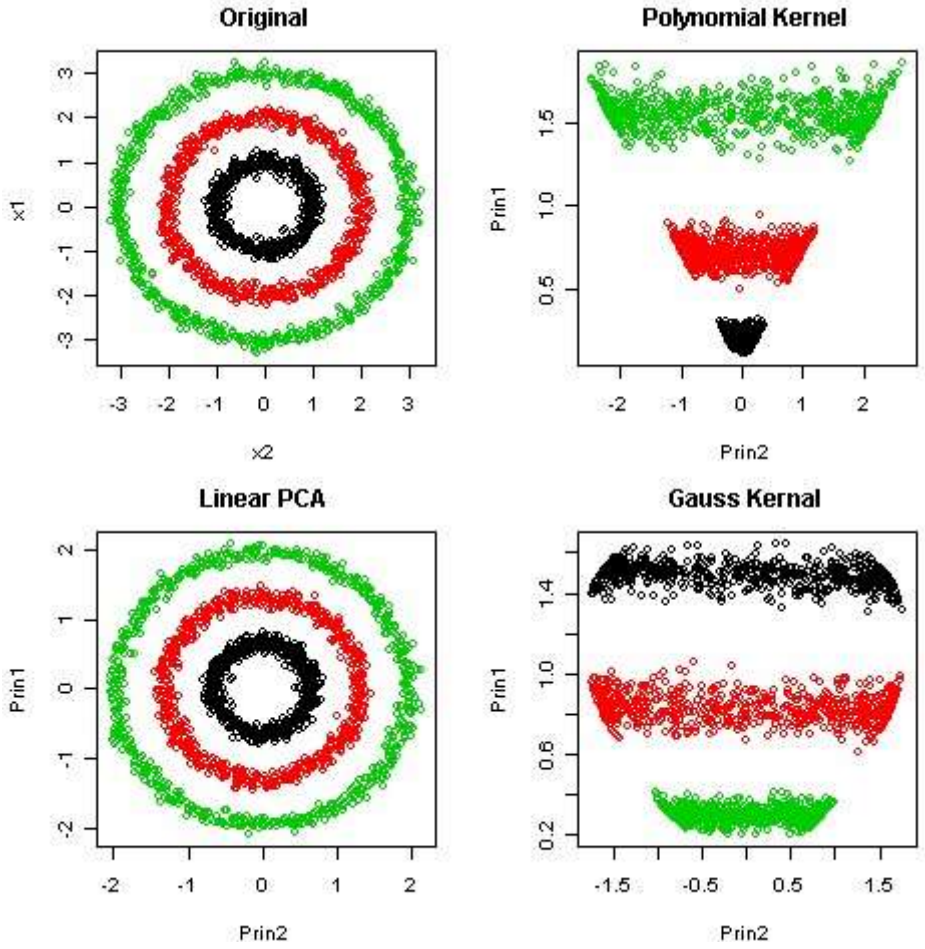
Kernel Principal Component Analysis (Kernel PCA) is an extension of Principal Component Analysis (PCA) that allows for the analysis of data that is not linearly separable. By using a kernel function to map data into a higher-dimensional space, Kernel PCA can uncover complex structures within the data that traditional PCA might miss.



PCA and Kernal PCA

How Kernel PCA Works

1. **Data Mapping with Kernel Trick**:

- Kernel PCA maps the original data into a higher-dimensional feature space using a kernel function.
- The kernel function k(xi,xj) computes the inner products between the images of the data points in the feature space, without explicitly computing the coordinates of the points in that space (the “kernel trick”).

2. Compute the Kernel Matrix:

- Construct a symmetric kernel matrix K where Kij=k(xi,xj).

3. Center the Kernel Matrix:

- Center the kernel matrix to ensure that the data has zero mean in the feature space. This is done using the formula:

$$K' = K - 1_N K - K 1_N + 1_N K 1_N$$

where 1_N is a matrix of ones divided by N (the number of data points).

4. Compute Eigenvalues and Eigenvectors:

- Perform eigenvalue decomposition on the centered kernel matrix K′. Let $\lambda_1, \lambda_2, \ldots, \lambda_N$ be the eigenvalues and $\alpha_1, \alpha_2, \ldots, \alpha_N$ be the corresponding eigenvectors.

5. Project Data onto Principal Components:

- The principal components are the projections of the data onto the eigenvectors in the feature space. The transformed data in the new feature space is given by:

$$\mathbf{z_i} = \sum_{j=1}^{N} \alpha_j k(x_i, x_j)$$

for each data point x_i.

Differences Between PCA and Kernel PCA

Linear vs. Non-Linear:

- PCA is a linear method and is only capable of capturing linear relationships in the data.
- Kernel PCA, on the other hand, can capture non-linear relationships by implicitly mapping the data into a higher-dimensional space using a kernel function.

Feature Space:

- In PCA, the data is transformed within the original feature space.

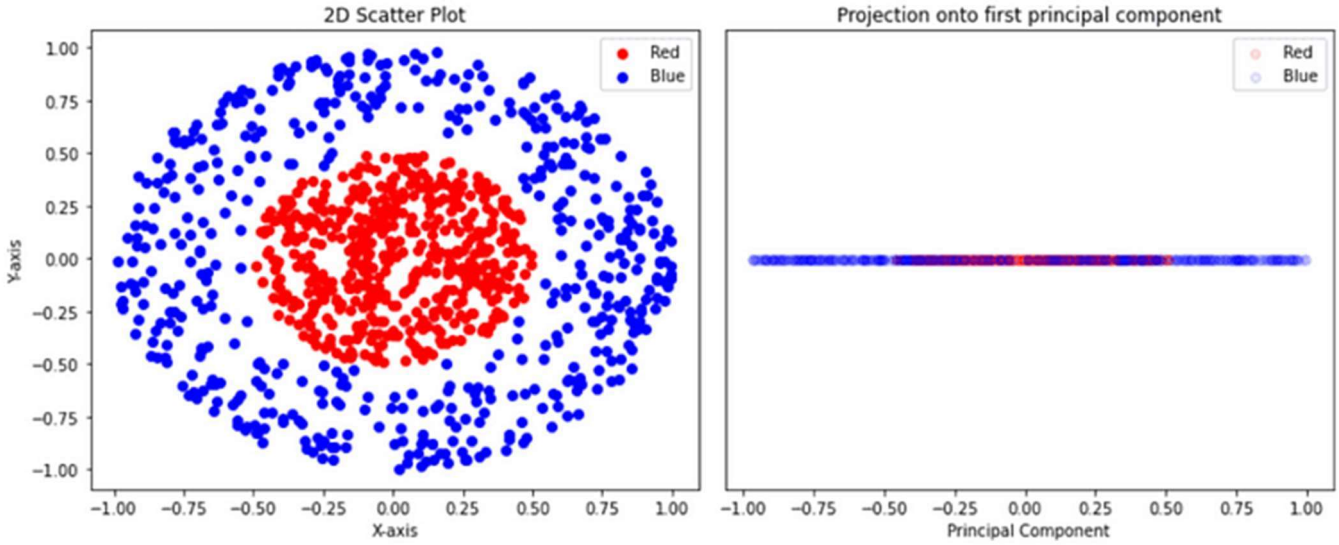
- In Kernel PCA, the data is transformed in a high-dimensional feature space determined by the kernel function.

Kernel Function:

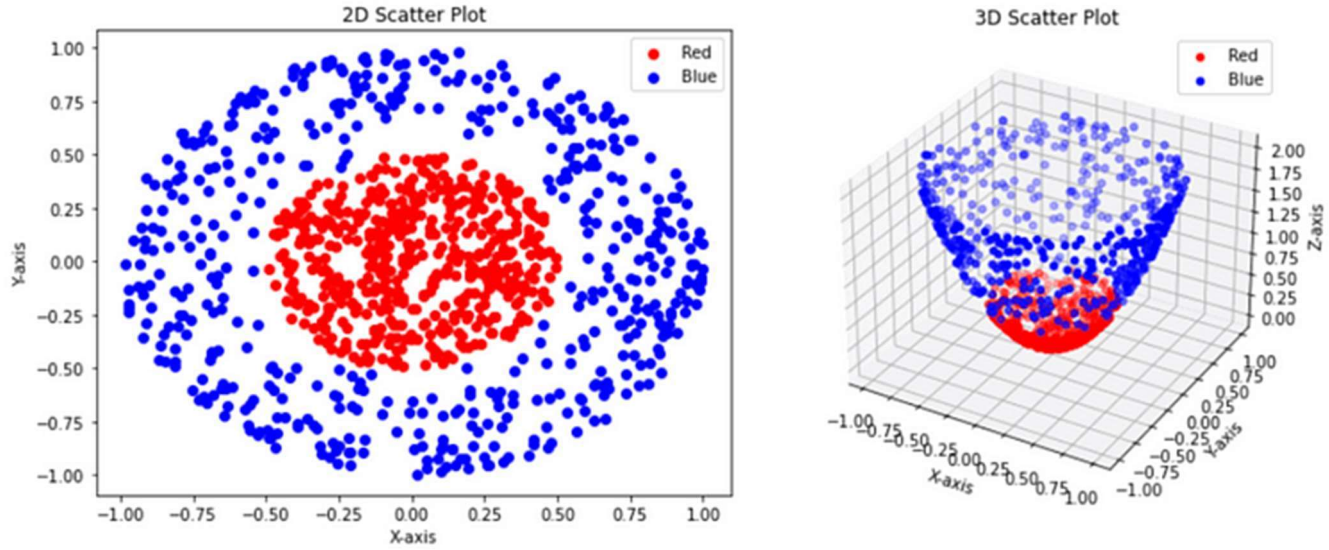
- PCA does not use a kernel function.
- Kernel PCA uses kernel functions (e.g., polynomial, Gaussian RBF) to compute the inner products in the high-dimensional feature space.

Working of Kernel PCA: Step-by-Step

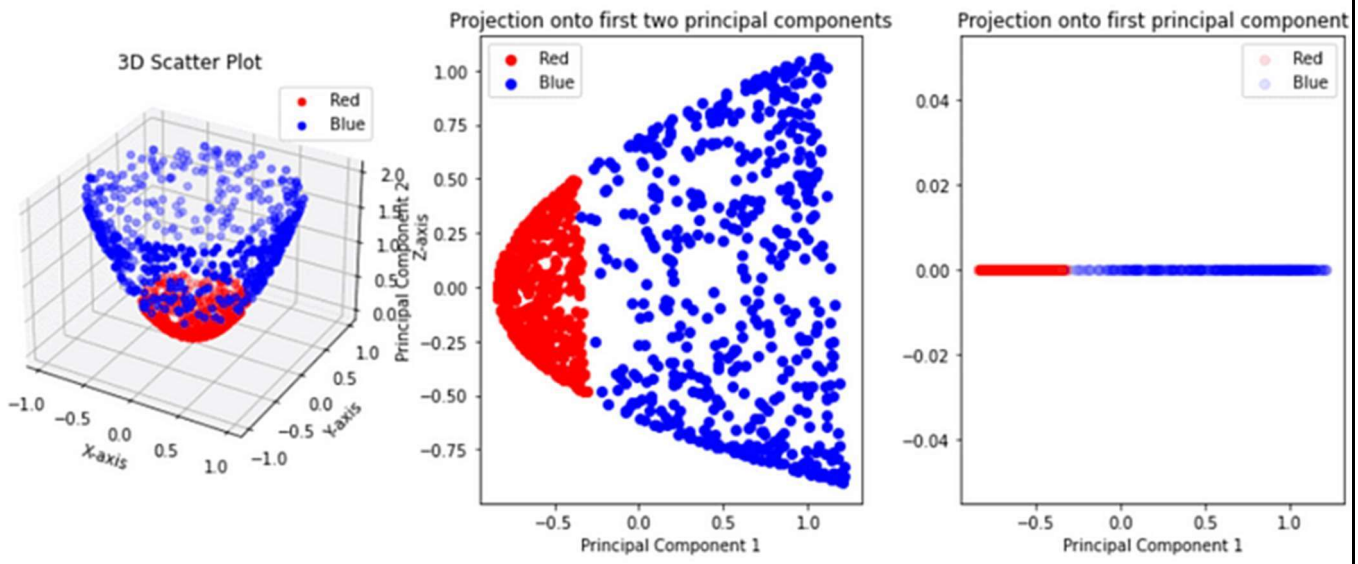
1. **Select a Kernel**: Choose a kernel function (e.g., polynomial, Gaussian RBF) based on the nature of the data and the problem at hand.
2. **Construct the Kernel Matrix**: Compute the kernel matrix KKK using the chosen kernel function for all pairs of data points.
3. **Center the Kernel Matrix**: Center the kernel matrix to have zero mean in the feature space.
4. **Eigenvalue Decomposition**: Perform eigenvalue decomposition on the centered kernel matrix to find the eigenvalues and eigenvectors.
5. **Project Data**: Project the original data onto the principal components (eigenvectors) in the high-dimensional feature space.



Data in 2D shows data is not linearly separable



Data in 3D



Now we can separate the data linearly.

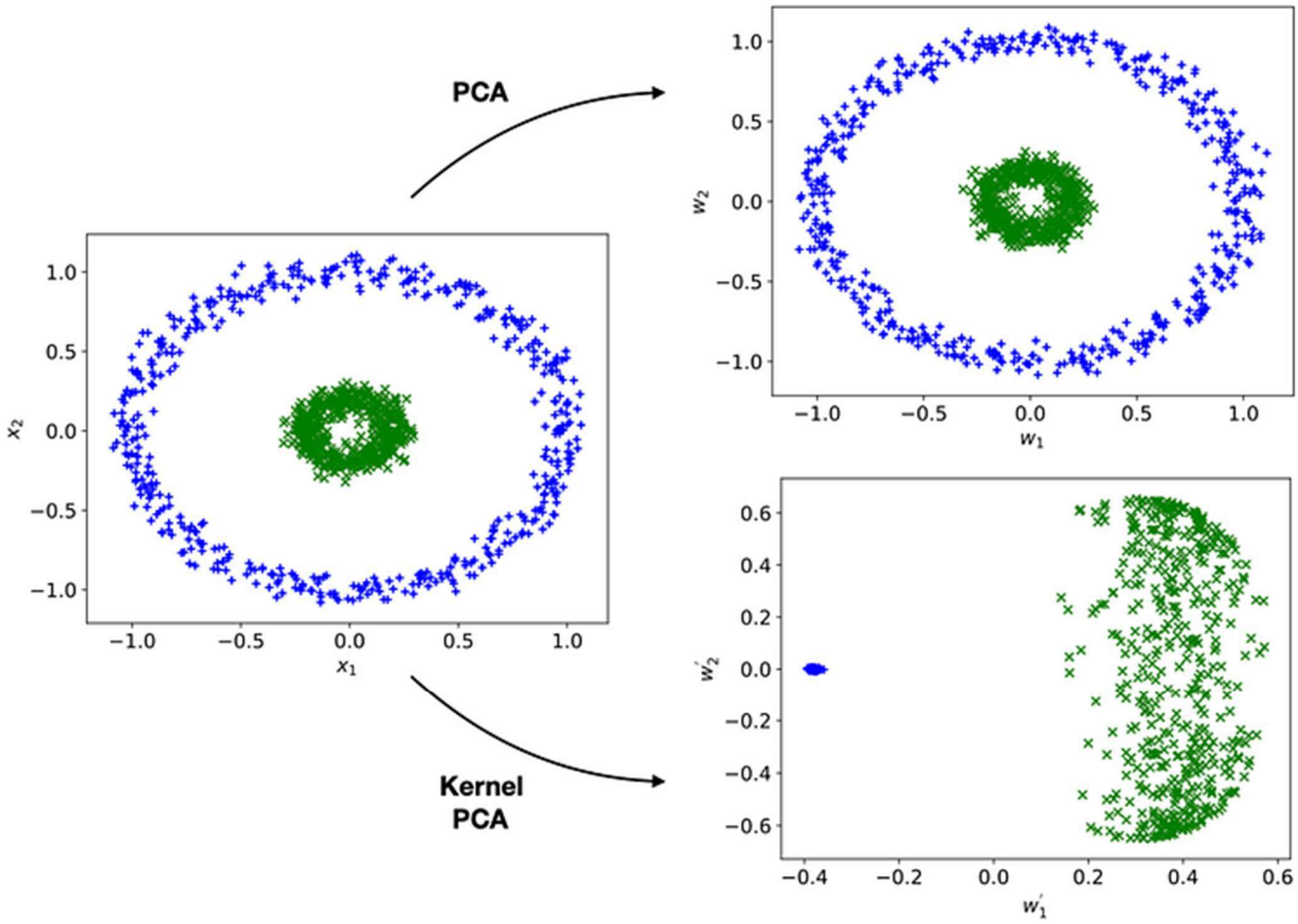
Pros and Cons of Kernel PCA

Pros:

- **Non-Linear Data**: Capable of handling non-linear data structures and capturing complex patterns.
- **Flexibility**: Various kernel functions can be used to adapt to different types of data and problems.
- **Higher Dimensional Insights**: Allows for the analysis of data in higher-dimensional spaces without explicitly computing the coordinates.

Cons:

- **Computationally Intensive**: Kernel PCA can be more computationally intensive than linear PCA, especially for large datasets.
- **Choice of Kernel**: The performance of Kernel PCA heavily depends on the choice of the kernel function and its parameters.
- **Interpretability**: The results of Kernel PCA can be harder to interpret compared to linear PCA, especially when using complex kernel functions.



PCA vs Kernal PCA

Common Kernel Functions in Kernel PCA

1. **Linear Kernel**
2. **Polynomial Kernel**
3. **Radial Basis Function (RBF) Kernel**
4. **Sigmoid Kernel**

When to Use Different Kernel Functions

- **Linear Kernel**: Use when the data is linearly separable or when you suspect that a linear relationship might be sufficient.
- **Polynomial Kernel**: Use when the data has polynomial relationships of a certain degree.
- **RBF Kernel**: Use when the data is not linearly separable, and you suspect the data can be separated with a non-linear boundary.
- **Sigmoid Kernel**: Use in cases where you want to mimic the behaviour of neural networks.

Kernel Type	Formula	Use Case	Pros	Cons
Linear Kernel	$k(x, y) = x \cdot y$	When data is linearly separable or close to linear separability.	Simple and fast; works well for linearly separable data.	Cannot capture non-linear relationships.
Polynomial Kernel	$k(x, y) = (x \cdot y + c)^d$	When data has polynomial relationships of a certain degree.	Can capture polynomial relationships; flexible with degree d.	Computationally expensive for high-degree polynomials; can overfit.
RBF Kernel (Gaussian Kernel)	$k(x, y) = \exp\left(-\frac{\|x-y\|^2}{2\sigma^2}\right)$	When data is not linearly separable; non-linear boundaries are needed.	Can capture very complex relationships; widely used and effective.	Requires careful tuning of σ; computationally intensive for large datasets.
Sigmoid Kernel	$k(x, y) = \tanh(\alpha x \cdot y + c)$	In scenarios mimicking neural network behavior.	Can capture certain non-linear relationships; inspired by neural networks.	Not as popular or effective as RBF in many scenarios; requires careful tuning of α and c.

Different types of Kernal PCA